Conceptos Fundamentales de Diseño y Optimización en PostgreSQL

Guía Teórico-Práctica para la Comprensión Interna del Motor y Defensa Técnica

---

1. Qué es la optimización y cuál es el problema real

Optimizar una base de datos no consiste en aplicar trucos mágicos ni en memorizar comandos. En términos reales de ingeniería, optimizar significa reducir el trabajo físico que debe realizar la computadora para resolver una consulta.

Cuando una base de datos procesa una petición, consume tres recursos de hardware concretos:

1. Operaciones de entrada y salida de disco (I/O): Es el recurso más lento y el principal cuello de botella. Leer datos desde el almacenamiento secundario toma órdenes de magnitud más tiempo que leerlos desde la memoria RAM.
2. Ciclos de procesador (CPU): El tiempo que pasa el procesador evaluando condiciones lógicas, realizando ordenamientos de registros en memoria o calculando funciones matemáticas y hashes para unir tablas.
3. Ancho de banda de red y memoria: La cantidad de bytes que viajan desde el proceso del motor de base de datos hacia la aplicación cliente que solicitó los datos.

Por lo tanto, una consulta optimizada es aquella que logra responder exactamente lo que el usuario necesita leyendo la menor cantidad posible de bloques de disco, utilizando la menor cantidad de memoria de trabajo y transfiriendo únicamente los datos indispensables.

---

2. La estructura física de PostgreSQL: Bloques (Block Size) y el Montículo (Heap)

Para entender por qué una base de datos se vuelve lenta, primero hay que comprender cómo guarda la información físicamente en el disco.

El Montículo (Heap)

En PostgreSQL, las tablas comunes se almacenan en una estructura denominada "Heap" o montículo. A diferencia de lo que uno podría intuir, las filas de una tabla dentro del Heap no se guardan ordenadas por clave primaria ni por fecha. Se van guardando en el orden en que van llegando, en cualquier espacio libre disponible dentro de los archivos del disco. El Heap es esencialmente una pila desordenada de registros.

El tamaño de bloque (Block Size o Page Size)

PostgreSQL no lee ni escribe registros individuales de a uno por vez en el disco. Toda la entrada y salida se realiza en unidades atómicas de tamaño fijo denominadas páginas o bloques.

- En PostgreSQL, el tamaño de bloque estándar es de 8 KB (8.192 bytes).

- Si una fila de la tabla `reserva` mide en promedio 70 bytes, dentro de una sola página de 8 KB van a entrar aproximadamente 100 o 110 filas.
- Cuando el motor necesita consultar un único registro, no puede traer 70 bytes del disco: tiene que cargar en memoria la página completa de 8 KB que contiene a ese registro.

Si la tabla `reserva` tiene 6.000.000 de registros, ocupa aproximadamente 55.000 a 57.000 páginas de 8 KB en disco (unos 440 MB solo de datos). Si no existe una forma de saber en qué página exacta está el dato buscado, el motor no tiene más opción que leer esas 57.000 páginas una por una desde el disco.

---

3. Qué es un Índice y qué problema resuelve

Un índice es una estructura de datos secundaria y auxiliar que se almacena separada del Heap. Su único objetivo es actuar como una guía de acceso rápido para encontrar filas sin tener que revisar la tabla entera.

La analogía tradicional y más clara es el índice temático al final de un libro técnico de mil páginas. Si estás buscando el concepto "Transacciones", no lees el libro desde la página 1 hasta la 1.000. Vas al índice temático, que está ordenado alfabéticamente, buscas la letra T, encuentras la palabra y el índice te dice exactamente: "Página 420". Luego vas directo a la página 420.

En PostgreSQL, el índice contiene dos cosas:

1. El valor de la columna indexada (por ejemplo, el ID del huésped o el estado de la reserva), ordenado de forma lógica.
2. Un puntero físico directo hacia el Heap, conocido como Tuple ID (`ctid`). El `ctid` indica exactamente: número de página de 8 KB y posición relativa de la fila dentro de esa página (por ejemplo: página 1.450, tupla 12).

La estructura B-Tree (Árbol B+)

Por defecto, PostgreSQL crea índices B-Tree (árboles balanceados). Esta estructura organiza los valores en una jerarquía de nodos:

- Nodo raíz: El punto de entrada en la cima del árbol.
- Nodos internos: Contienen rangos de valores que guían la búsqueda hacia abajo.
- Nodos hoja: La base del árbol, donde están los valores reales ordenados y sus punteros al Heap. Además, las hojas están unidas entre sí como una lista doblemente enlazada, lo que permite recorrer rangos de valores hacia adelante o hacia atrás de forma muy eficiente.

En una tabla de 6 millones de filas, un árbol B-Tree tiene una profundidad de apenas 3 o 4 niveles. Esto significa que con solo descender por 3 o 4 páginas de índice, el motor localiza cualquier registro entre millones, reduciendo el costo de buscar entre 57.000 bloques a leer 4 o 5 bloques.

---

4. Métodos de acceso del motor: Cómo lee los datos PostgreSQL

Al analizar consultas con el planificador, se observan distintos métodos de acceso físico. Conocer la diferencia entre ellos es fundamental.

1. Escaneo Secuencial (Sequential Scan / Seq Scan)

Es el método más básico y directo: el motor abre el archivo de la tabla en disco y lee absolutamente todas las páginas de 8 KB de principio a fin, evaluando la condición fila por fila.

- Cuándo es bueno: En tablas chicas (de pocas páginas) o cuando la consulta pide una porción muy grande de la tabla (por ejemplo, el 70% de las filas), ya que leer el disco de forma continua y secuencial es muy veloz mecánicamente.
- Cuándo es malo: En tablas medianas o grandes donde buscamos una aguja en un pajar (por ejemplo, buscar 1 reserva específica entre 6 millones). En ese caso, leer 57.000 bloques para traer 1 sola fila es un desperdicio masivo de recursos.

2. Escaneo por Índice (Index Scan)

El motor recorre el árbol B-Tree buscando el valor filtrado, obtiene el puntero `ctid` de cada coincidencia y, de inmediato, va al Heap a buscar los datos completos de esa fila.

- Cuándo se usa: Cuando el filtro es muy selectivo (encuentra pocas filas, por ejemplo entre 1 y unos pocos cientos).
- Desventaja: Cada salto desde el índice hacia el Heap es una lectura aleatoria de disco. Si hay muchas coincidencias, hacer miles de saltos aleatorios termina siendo más lento que leer la tabla completa de corrido.

3. Escaneo por Mapa de Bits de Índice (Bitmap Index Scan + Bitmap Heap Scan)

Es una estrategia intermedia ideada por PostgreSQL para evitar los saltos aleatorios en disco:

1. Fase 1 (Bitmap Index Scan): Recorre el índice y, en lugar de ir al Heap inmediatamente por cada fila, arma un mapa de bits en memoria RAM. Este mapa marca con un "1" cada página física de 8 KB que contiene al menos una fila coincidente.
2. Fase 2 (Bitmap Heap Scan): Toma ese mapa de bits, ordena las páginas físicas por su ubicación secuencial en el disco, y visita cada página una sola vez en orden, eliminando saltos aleatorios.

- Cuándo se usa: Cuando la consulta devuelve una cantidad intermedia de filas (por ejemplo, entre el 1% y el 10% de la tabla).

4. Escaneo de Solo Índice (Index Only Scan)

Es el escenario más rápido posible en el motor. Ocurre cuando todas las columnas solicitadas en la consulta (tanto en el `SELECT` como en el `WHERE`) ya están presentes dentro del propio índice.

- El motor lee únicamente las páginas del índice y jamás necesita tocar las páginas del Heap.
- El indicador clave en el plan de ejecución es `Heap Fetches: 0`. Esto significa que no se realizó ningún acceso al montículo.

---

5. El Optimizador de Costos y el Punto de Inflexión (Tipping Point)

Una duda muy común de un desarrollador trainee es: "*Le creé un índice a la columna, pero cuando ejecuto la consulta, PostgreSQL sigue haciendo Seq Scan. ¿El motor está roto?*".

La respuesta es no. PostgreSQL utiliza un Optimizador Basado en Costos (CBO). Antes de ejecutar cualquier consulta, el motor calcula matemáticamente el costo estimado de todos los caminos posibles (usar el índice,

usar un escaneo secuencial, usar un mapa de bits) y elige el que tenga menor costo abstracto.

El costo abstracto (`cost=inicio..total`)

El costo no se mide en segundos ni en milisegundos, sino en unidades arbitrarias de esfuerzo de hardware configuradas en el servidor:

- `seq_page_cost = 1.0`: Costo de leer una página de 8 KB de forma secuencial.
- `random_page_cost = 4.0`: Costo de leer una página de 8 KB mediante un salto aleatorio. Por defecto se asume que un salto aleatorio cuesta 4 veces más que una lectura secuencial.
- `cpu_tuple_cost = 0.01`: Costo de procesar una fila en CPU.

El Punto de Inflexión (Tipping Point)

Si una consulta filtra por `estado = 'FINALIZADA'` y el 65% de las 6.000.000 de reservas están finalizadas (3,9 millones de filas), el motor calcula:

- Si usa el índice: Debe hacer casi 4 millones de lecturas aleatorias en el Heap (`random_page_cost`), saltando de un lado al otro del disco.
- Si usa Seq Scan: Lee las 57.000 páginas de corrido (`seq_page_cost`) en un solo barrido secuencial continuo y aplica el filtro en procesador.

El cálculo matemático demuestra que leer 57.000 páginas secuenciales es mucho más barato que hacer 3.900.000 saltos aleatorios. Por lo tanto, cuando una condición devuelve más del 5% al 15% de la tabla (baja selectividad), el optimizador inteligentemente descarta el índice y elige un Sequential Scan. El índice solo se usa cuando la condición es selectiva (recupera una porción pequeña de los datos).

---

6. Diagnóstico con EXPLAIN y EXPLAIN ANALYZE

Para saber qué está haciendo el motor frente a una consulta, se utiliza `EXPLAIN`.

La diferencia entre EXPLAIN y EXPLAIN ANALYZE

- `EXPLAIN`: Le pide al planificador que muestre su estrategia estimada sin ejecutar la consulta. Calcula costos estimados (`cost=...`) y filas estimadas (`rows=...`) basándose en las estadísticas de las tablas.
- `EXPLAIN ANALYZE`: Planifica la consulta, la ejecuta físicamente en el motor hasta el final, y muestra las métricas reales observadas durante la ejecución: tiempo real en milisegundos (`actual time`), filas reales procesadas (`actual rows`), número de vueltas (`loops`) y memoria consumida.

Qué significa cada número en la salida

Ejemplo típico:

```
Index Scan using reserva_pkey on reserva (cost=0.43..8.45 rows=1 width=30) (actual
time=0.086..0.094 rows=20 loops=1)
```

1. `cost=0.43..8.45`: El primer número (0.43) es el costo de arranque (tiempo estimado hasta que el nodo puede escupir la primera fila). El segundo número (8.45) es el costo total estimado para devolver

todas las filas de ese paso.
2. `rows=1`: Cantidad de filas que el planificador estimaba que iban a coincidir.
3. `width=30`: Ancho promedio estimado en bytes de cada fila devuelta por ese nodo.
4. `actual time=0.086..0.094`: Tiempo real medido en milisegundos. El primer número es cuándo salió la primera fila; el segundo es cuándo terminó el nodo.
5. `actual rows=20`: Cantidad real de filas que procesó el nodo. Si hay una discrepancia enorme entre `rows` estimado (ej: 1) y `actual rows` real (ej: 50.000), significa que las estadísticas de la tabla están desactualizadas y hace falta correr `ANALYZE`.

El parámetro BUFFERS

Al agregar `BUFFERS` (`EXPLAIN (ANALYZE, BUFFERS)`), el motor expone el consumo real de páginas de 8 KB:

- `shared hit`: Cantidad de páginas que ya estaban cargadas en la memoria RAM del servidor (`shared_buffers`) y se leyeron a velocidad de memoria.
- `shared read`: Cantidad de páginas que no estaban en memoria y el motor tuvo que ir a pedirlas físicamente al disco duro o SSD.
- `temp read / temp written`: Cantidad de páginas que la consulta tuvo que escribir y leer en disco temporal porque la memoria asignada (`work_mem`) no alcanzó para procesar un ordenamiento o un hash join. Si estos números son mayores a cero, la consulta está sufriendo un cuello de botella grave de memoria.

---

7. Memoria y Caché: Shared Buffers, Hit Ratio, Cold Cache vs Warm Cache

PostgreSQL administra un área de memoria RAM compartida entre todos los procesos de clientes llamada `shared_buffers`.

Cómo opera el búfer de páginas

Cuando una consulta necesita leer una fila, el motor busca primero si la página de 8 KB donde vive esa fila ya reside dentro de `shared_buffers`.

- Si la página ya está en RAM, se registra un `shared hit`. El dato se lee casi instantáneamente (nanosegundos o microsegundos).
- Si la página no está en RAM, se registra un `shared read`. El proceso debe solicitar la página al sistema operativo o al disco físico, guardarla en `shared_buffers` y recién ahí procesarla.

Cold Cache (Caché Fría) vs Warm Cache (Caché Caliente)

- Cold Cache: Ocurre cuando el servidor recién arranca o hace mucho que nadie consulta esa tabla. La memoria RAM está vacía de esos datos. La primera vez que se ejecuta una consulta pesada, la mayoría de los bloques se leen físicamente del disco (`shared read`), por lo que la consulta puede tardar, por ejemplo, 1.200 milisegundos.
- Warm Cache: Ocurre cuando la misma consulta (o una consulta similar sobre la misma tabla) se ejecuta por segunda vez consecutiva. Las páginas leídas en la primera ejecución quedaron retenidas en la memoria RAM (`shared_buffers` y caché de páginas del sistema operativo). En esta segunda vuelta, el

motor no va al disco: lee directamente de RAM (`shared hit`), reduciendo el tiempo de 1.200 ms a 15 o 30 ms sin haber tocado una sola línea de código SQL.

Cache Hit Ratio (Tasa de Acierto de Caché)

Es el porcentaje de lecturas que se resolvieron directamente en memoria sin acudir al disco: $$\text{Hit Ratio} = \frac{\text{shared_hit}}{\text{shared_hit} + \text{shared_read}} \times 100$$ En un entorno de producción saludable, la tasa de acierto de caché de una base de datos debe ser superior al 95% en tablas y superior al 98% en índices.

---

8. SARGabilidad: Cómo escribir filtros que el motor pueda aprovechar

El término SARGable proviene del inglés *Search Argument Able* (capaz de ser utilizado como argumento de búsqueda).

Un predicado en una cláusula `WHERE` es SARGable cuando permite que el optimizador utilice un índice B-Tree para saltar directamente a los registros deseados mediante una búsqueda binaria en $O(\log N)$.

La regla de oro de la SARGabilidad

Para que una condición sea SARGable, la columna indexada debe estar completamente sola en uno de los lados del operador de comparación (`=`, `<`, `>`, `<=`, `>=`, `BETWEEN`). No puede estar envuelta en funciones matemáticas, funciones de texto, concatenaciones, cálculos de fechas ni conversiones de tipo.

Ejemplos clásicos de consultas arruinadas vs optimizadas

Ejemplo A: Cálculos con fechas

- No SARGable:

  ```
  WHERE (fecha_inicio + INTERVAL '10 days') >= '2025-01-01'
  ```

  El motor tiene que tomar cada una de las 6.000.000 de filas, sumarle 10 días al valor de la columna en CPU, y recién comparar con la fecha. Esto destruye la propiedad de ordenamiento del B-Tree y fuerza un barrido secuencial masivo de 57.000 bloques.
- SARGable (Transposición a constante):

  ```
  WHERE fecha_inicio >= '2024-12-22'
  ```

  La columna quedó limpia. El cálculo de los 10 días se le restó directamente a la fecha constante antes de ejecutar. El motor toma '2024-12-22', desciende por las 3 ramas del B-Tree y entra directo a las filas en milisegundos.

Ejemplo B: Extracción de componentes

- No SARGable:

  ```
  WHERE EXTRACT(YEAR FROM fecha_inicio) = 2024
  ```

  Evalúa la función `EXTRACT` tupla por tupla en 6 millones de registros.
- SARGable:

  ```
  WHERE fecha_inicio >= '2024-01-01' AND fecha_inicio < '2025-01-01'
  ```

  Define un rango cerrado/semiabierto directo sobre la columna indexada.

Ejemplo C: Funciones de texto

- No SARGable:

  ```
  WHERE UPPER(apellido) = 'PEREZ'
  ```

  A menos que exista un índice funcional específico, el índice normal sobre `apellido` no sirve porque el árbol está ordenado con los textos tal cual fueron insertados, no en mayúsculas.

---

9. Estrategias de indexación avanzadas

Crear un índice simple sobre una sola columna resuelve las búsquedas básicas, pero los sistemas reales demandan estrategias más sofisticadas.

1. Índices Compuestos y la Regla del Prefijo Izquierdo

Un índice compuesto se crea sobre dos o más columnas, por ejemplo:

```
CREATE INDEX idx_reserva_estado_fecha ON reserva(estado, fecha_inicio);
```

El árbol B-Tree se ordena primariamente por la primera columna (`estado`) y, dentro de cada estado idéntico, se ordena secundariamente por la segunda columna (`fecha_inicio`).

- La Regla del Prefijo Izquierdo establece que el índice solo puede utilizarse si la consulta filtra por la columna que encabeza el índice (la columna izquierda), o por ambas columnas juntas.
  - `WHERE estado = 'CONFIRMADA' AND fecha_inicio >= '2025-01-01'`: Usa el índice al 100% (ambas columnas).
  - `WHERE estado = 'CONFIRMADA'`: Usa el índice eficientemente (aprovecha la columna líder).
  - `WHERE fecha_inicio >= '2025-01-01'`: El optimizador descarta el índice compuesto. Como el árbol está agrupado primero por estado, las fechas están dispersas en distintas ramas y el motor no puede descender binariamente solo por fecha.

2. Índices de Cobertura con cláusula INCLUDE

Tradicionalmente, si querías que una consulta no fuera al Heap, tenías que poner todas las columnas dentro de la clave del índice. El problema es que indexar columnas adicionales agranda el árbol y encarece las comparaciones.

La cláusula `INCLUDE` separa la clave de búsqueda de la carga útil:

```
CREATE INDEX idx_reserva_cobertura ON reserva (id_huesped, estado) INCLUDE
(tarifa);
```

- `id_huesped` y `estado` forman la estructura de ordenamiento del B-Tree (nodos raíz e internos).
- `tarifa` no participa del ordenamiento: simplemente se almacena como dato extra en los nodos hoja.
- Resultado: Una consulta que pide `SELECT id_huesped, estado, SUM(tarifa) ... WHERE id_huesped = 4 AND estado = 'CONFIRMADA'` encuentra todo lo que necesita en las hojas del índice. No hace ningún salto al Heap (`Heap Fetches: 0`), logrando un `Index Only Scan` ultra rápido.

3. Índices Funcionales

Si la lógica de negocio obliga a buscar texto ignorando mayúsculas y minúsculas, se crea un índice sobre el resultado de la función:

```
CREATE INDEX idx_huesped_apellido_lower ON huesped (LOWER(apellido));
```

El B-Tree no guarda el apellido original, guarda el valor precalculado en minúsculas. Cuando el usuario busca `WHERE LOWER(apellido) = 'perez'`, el motor compara directamente contra el índice funcional sin evaluar la función fila por fila.

4. Índices GiST para rangos de fechas (Exclusión sin Triggers)

En un sistema hotelero, dos reservas para una misma habitación no pueden solaparse en el tiempo. Resolver esto con un trigger que haga un `SELECT COUNT(*)` antes de cada `INSERT` es ineficiente y susceptible a condiciones de carrera en alta concurrencia.

PostgreSQL ofrece el tipo de datos `daterange` y los índices GiST (Generalized Search Tree):

```
ALTER TABLE reserva ADD CONSTRAINT no_solapamiento
EXCLUDE USING gist (id_habitacion WITH =, daterange(fecha_inicio, fecha_salida,
'[]') WITH &&);
```

El operador `&&` verifica la superposición geométrica/temporal de intervalos en tiempo logarítmico. Si entra una reserva que solapa con una existente en la misma habitación, el propio motor la rechaza a nivel físico instantáneamente, garantizando integridad estricta sin bloqueos de aplicación.

---

10. El modelo MVCC, Tuplas Muertas, Hinchamiento (Bloat) y VACUUM

PostgreSQL implementa el Control de Concurrencia Multiversión (MVCC). Su principio fundacional es: "*Las lecturas jamás bloquean a las escrituras, y las escrituras jamás bloquean a las lecturas*".

Qué pasa físicamente en un UPDATE o DELETE

En otros motores de base de datos tradicionales, modificar una fila sobreescribe los bytes en el disco. En PostgreSQL no es así:

- Cuando ejecutas un `DELETE`, la fila no se borra físicamente. Se marca internamente con un identificador de transacción de borrado (`xmax`). A partir de ese momento, la fila pasa a ser invisible para nuevas transacciones, pero sigue ocupando espacio en el bloque de 8 KB. Se convirtió en una tupla muerta (*dead tuple*).
- Cuando ejecutas un `UPDATE`, PostgreSQL ejecuta internamente un borrado lógico de la fila anterior (convirtiéndola en tupla muerta) y realiza un `INSERT` de la fila nueva con los valores modificados en otro lugar de la página o en otra página distinta.

El problema del Hinchamiento (Bloat)

Si en una tabla de 6 millones de filas se actualiza el estado de 500.000 reservas, la tabla ahora tiene 6.000.000 de tuplas vivas y 500.000 tuplas muertas. Las tuplas muertas siguen ocupando espacio en las páginas de disco. Con el paso de los meses, una tabla de 500 MB puede terminar ocupando 3 GB de páginas llenas de registros muertos que ralentizan todos los escaneos. A este fenómeno se lo denomina "Bloat" o hinchamiento.

El rol de VACUUM

`VACUUM` es el recolector de basura (*Garbage Collector*) de PostgreSQL. Existen tres variantes fundamentales:

1. `VACUUM` (conocido como Lazy Vacuum):
    - Recorre las páginas de la tabla, busca las tuplas muertas que ya no son visibles para ninguna transacción activa y marca ese espacio como "disponible" para que futuros `INSERT` escriban encima.
    - No devuelve el espacio libre al sistema operativo (el archivo `.ibd` o de tabla no se achica en Windows o Linux).
    - No bloquea la tabla: las aplicaciones pueden seguir leyendo y escribiendo con normalidad.
2. `VACUUM FULL`:
    - Crea un archivo completamente nuevo en disco, copia únicamente las tuplas vivas una al lado de la otra de forma compacta y borra el archivo viejo inflado.
    - Sí devuelve el espacio sobrante al sistema operativo.
    - Peligro crítico: Adquiere un candado exclusivo absoluto (`AccessExclusiveLock`). Mientras corre `VACUUM FULL`, nadie puede ni leer ni escribir en la tabla. En una tabla de millones de filas puede tardar minutos u horas, dejando la aplicación fuera de servicio. No se debe usar en producción rutinaria.
3. `VACUUM ANALYZE`:

- Ejecuta la limpieza de tuplas muertas y, a continuación, ejecuta `ANALYZE`.
- `ANALYZE` recorre una muestra representativa de la tabla y actualiza las estadísticas del catálogo interno (`pg_statistic` / `pg_stats`): calcula la cantidad real de filas, cuántos valores distintos hay (`n_distinct`), los valores más frecuentes (`most_common_vals`) y la correlación física en disco.
- Sin `ANALYZE`, el optimizador toma decisiones basadas en datos viejos y puede elegir un plan pésimo.

Autovacuum

Es un proceso daemon en segundo plano que corre automáticamente. Monitorea las tablas y, cuando la cantidad de tuplas muertas supera un umbral porcentual configurable (`autovacuum_vacuum_scale_factor`, por defecto 20%), dispara un `VACUUM ANALYZE` en segundo plano sin intervención humana.

---

11. Afinamiento de memoria del servidor (`postgresql.conf`)

El archivo de configuración principal de PostgreSQL gobierna cómo el motor reparte los recursos de hardware del equipo.

1. `shared_buffers`

- Qué es: La memoria RAM dedicada para almacenar en caché las páginas de datos e índices más consultados.
- Regla estándar: En un servidor dedicado a base de datos, se configura aproximadamente en el 25% de la memoria RAM total del sistema (por ejemplo, 4 GB en una máquina de 16 GB).
- Por qué no poner el 80%: Porque PostgreSQL no compite con el sistema operativo; trabaja en conjunto con él. El sistema operativo utiliza la memoria RAM restante como *Page Cache* del kernel. Si se le da demasiada memoria a PostgreSQL, se produce una doble paginación ineficiente.

2. `work_mem`

- Qué es: La memoria asignada para operaciones internas de ordenamiento (`ORDER BY`, `DISTINCT`), agregaciones y tablas hash para `JOIN`.
- Comportamiento crítico: `work_mem` no es una cantidad fija por usuario ni por consulta. Es una cantidad asignada por cada nodo de operación individual dentro de una consulta.
  - Si una consulta compleja ejecuta 3 Hash Joins y 2 Sorts, esa sola consulta puede reservar $5 \times \text{work_mem}$.
  - Si hay 30 conexiones simultáneas ejecutando consultas similares con `work_mem = 64MB`, el consumo puede alcanzar $30 \times 5 \times 64\text{ MB} = 9.600\text{ MB}$ (casi 10 GB de RAM).
  - Si se configura un `work_mem` excesivo, el servidor se queda sin RAM física y el sistema operativo activa el OOM Killer (Out-Of-Memory Killer), matando intempestivamente el proceso del motor.

3. `maintenance_work_mem`

- Qué es: Memoria utilizada únicamente para tareas pesadas de mantenimiento administrativo: `VACUUM`, `CREATE INDEX`, `REINDEX` y agregado de claves foráneas.

- Comportamiento: Como estas operaciones las ejecuta un administrador de a una por vez, se puede configurar un valor generoso (ej: 512 MB o 1 GB). Cuanta más memoria tenga `maintenance_work_mem`, más rápido se ordenan los datos en RAM para construir un índice nuevo sin volcar datos temporales al disco.

4. `max_connections` y Connection Pooling

- Cada conexión que se abre contra PostgreSQL levanta un proceso independiente en el sistema operativo mediante la llamada `fork()`. Cada proceso consume entre 5 y 10 MB de memoria solo por existir.
- Abrir 1.000 conexiones directas consume varios gigabytes de memoria sin haber ejecutado una sola consulta y satura el planificador del sistema operativo haciendo cambios de contexto (*context switching*).
- Solución arquitectónica: Mantener `max_connections` en un número moderado (ej: 100 a 200) y colocar adelante un Connection Pooler como **PgBouncer**. PgBouncer mantiene un grupo pequeño de conexiones permanentes contra el motor y atiende a miles de clientes web multiplexando las transacciones de forma ultra eficiente.

---

12. Concurrencia, Bloqueos y Deadlocks

Cuando múltiples usuarios consultan y modifican los mismos datos al mismo tiempo, el motor debe garantizar la consistencia sin frenar la operatoria.

Bloqueos de fila vs Bloqueos de tabla

- Bloqueo de fila (`SELECT ... FOR UPDATE`): Bloquea exclusivamente las tuplas individuales seleccionadas para una transacción de actualización. Otras sesiones pueden seguir leyendo esas filas (ven la versión previa por MVCC) y pueden modificar cualquier otra fila de la misma tabla sin ninguna interferencia.
- Bloqueo exclusivo de tabla (`LOCK TABLE ... IN ACCESS EXCLUSIVE MODE`): Congela la tabla entera. Nadie puede leer ni escribir hasta que la transacción termine. Se utiliza casi exclusivamente para operaciones DDL (como `ALTER TABLE` que cambian la estructura física).

Qué es un Deadlock (Interbloqueo Mutuo)

Un deadlock ocurre cuando dos o más transacciones quedan atrapadas en un ciclo de espera circular cerrado donde ninguna puede avanzar porque la otra retiene el recurso que necesita:

- La Transacción 1 tiene bloqueada la Habitación 1 y solicita bloquear la Habitación 2.
- Al mismo tiempo, la Transacción 2 tiene bloqueada la Habitación 2 y solicita bloquear la Habitación 1.
- La Transacción 1 espera a la Transacción 2; la Transacción 2 espera a la Transacción 1 ($T_1 \rightarrow T_2 \rightarrow T_1$).

Cómo actúa PostgreSQL frente a un Deadlock

Si el motor no hiciera nada, ambas sesiones quedarían congeladas para siempre. PostgreSQL tiene un temporizador interno gobernado por el parámetro `deadlock_timeout` (configurado por defecto en 1 segundo).

1. Cuando una sesión se queda esperando un bloqueo por más de 1 segundo, el motor activa el algoritmo de detección de ciclos en el grafo de bloqueos.
2. Si detecta el ciclo cerrado, PostgreSQL elige arbitrariamente una de las dos transacciones y la aborta de inmediato emitiendo un error:

   ```
   ERROR: deadlock detected
   DETAIL: Process 5410 waits for ShareLock on transaction 89201; blocked by
   process 5412.
   SQL state: 40P01
   ```

3. Al abortarse una de las sesiones, se liberan sus candados y la otra transacción puede continuar normalmente.

- Prevención en desarrollo: Para evitar deadlocks, las aplicaciones deben adquirir siempre los recursos en el mismo orden determinístico (por ejemplo, ordenando los IDs antes de bloquear: `ORDER BY id`).

Diagnóstico en tiempo real

Para ver qué sesión está frenando a otra, PostgreSQL ofrece la función nativa `pg_blocking_pids()`:

```
SELECT pid, pg_blocking_pids(pid) AS bloqueado_por_pid, state, wait_event, query
FROM pg_stat_activity
WHERE cardinality(pg_blocking_pids(pid)) > 0;
```

Permite identificar al instante el proceso culpable y, si es necesario, cancelarlo de forma controlada mediante `SELECT pg_cancel_backend(pid)` o `SELECT pg_terminate_backend(pid)`.

---

13. Escalabilidad física: Particionamiento y Partition Pruning

A medida que una tabla transaccional supera los millones de registros, los índices B-Tree se vuelven tan grandes que ya no entran en la memoria RAM (`shared_buffers`), obligando a leer páginas del índice desde el disco.

Particionamiento declarativo por rango

El particionamiento consiste en dividir lógicamente una tabla masiva única en múltiples tablas hijas físicas más pequeñas organizadas por una clave (habitualmente una fecha):

- La tabla madre `reserva_particionada` actúa como fachada transparente para las aplicaciones.
- Físicamente existen particiones separadas: `reserva_p2023`, `reserva_p2024`, `reserva_p2025` y una partición por defecto `reserva_pdefault`.
- Cada partición tiene su propio archivo físico independiente y sus propios índices B-Tree compactos que caben holgadamente en la memoria RAM.

Partition Pruning (Poda de Particiones)

Es la capacidad del optimizador de descartar tablas hijas completas antes de ejecutar la consulta. Si la aplicación ejecuta:

```
SELECT * FROM reserva_particionada
WHERE fecha_inicio >= '2024-06-01' AND fecha_inicio < '2024-07-01';
```

El optimizador inspecciona los metadatos de las restricciones de rango en tiempo de planificación y detecta que ese intervalo solo puede existir dentro de `reserva_p2024`.

- Automáticamente "poda" (ignora por completo) las particiones de 2023, 2025 y por defecto.
- El plan de ejecución muestra que el motor solo abrió y leyó `reserva_p2024`, descartando el 90% del volumen de datos sin haber tenido que leer ni un solo bloque de las otras particiones.

Purgas históricas en tiempo constante $O(1)$

En una tabla gigante tradicional, eliminar las reservas viejas de hace 5 años requiere un `DELETE FROM reserva WHERE fecha_inicio < '2020-01-01'`. Esa sentencia genera millones de tuplas muertas, satura el log transaccional (WAL), demora horas y genera un bloat inmenso. En una tabla particionada, purgar un año entero se resuelve en milisegundos desconectando la tabla hija:

```
ALTER TABLE reserva_particionada DETACH PARTITION reserva_p2020;
DROP TABLE reserva_p2020;
```

Es una operación de catálogo en $O(1)$: elimina el archivo físico de disco de inmediato, sin generar una sola tupla muerta y sin consumir CPU.

---

14. Vistas tradicionales frente a Vistas Materializadas

La Vista Tradicional (`CREATE VIEW`)

Una vista común no almacena datos en disco. Es simplemente una consulta SQL guardada con un nombre. Cada vez que un usuario hace `SELECT * FROM vista_reporte_gerencial`:

- El motor sustituye el nombre de la vista por la consulta subyacente.
- Si la vista involucra cruzar 5 tablas relacionales y calcular sumas sobre 6 millones de filas, ese cruce masivo se ejecuta íntegramente desde cero cada vez que alguien entra a la pantalla, demorando cientos de milisegundos o varios segundos.

La Vista Materializada (`CREATE MATERIALIZED VIEW`)

Una vista materializada ejecuta la consulta una sola vez y guarda el resultado físicamente en una tabla real en el disco.

- Si el cálculo de facturación por hotel sobre 6 millones de reservas demoraba 800 ms, la vista materializada condensa el resultado en apenas 200 filas guardadas en disco.
- Al ser una tabla real, permite crearle **sus propios índices B-Tree**:

```
CREATE UNIQUE INDEX uq_matview_hotel ON matview_facturacion_hotel(hotel_id);
```

- Cuando el usuario consulta la vista materializada indexada, el motor no toca las 6 millones de reservas: lee directamente la tabla precalculada en **0,06 ms**, resultando más de 10.000 veces más rápida.

Refresco Concurrente no bloqueante

El dato de una vista materializada es una foto estática en el tiempo; si entran nuevas reservas, la vista no se actualiza sola. Para actualizar los datos sin congelar las consultas de los usuarios se utiliza el refresco concurrente:

```
REFRESH MATERIALIZED VIEW CONCURRENTLY matview_facturacion_hotel;
```

Gracias al índice único, PostgreSQL calcula las diferencias en segundo plano e intercambia los registros modificados sin adquirir bloqueos exclusivos, permitiendo que los reportes gerenciales sigan respondiendo en tiempo real mientras la información se actualiza.

---

15. Glosario Rápido para Respuestas Claras en la Defensa

- **SARGable**: Predicado que deja la columna limpia sin funciones para permitir búsqueda binaria en un B-Tree.
- **Heap**: El archivo desordenado donde residen físicamente las tuplas de una tabla.
- **ctid**: El puntero físico compuesto por (número de página, índice de tupla) que apunta al Heap.
- **Block Size**: Bloque básico de entrada/salida de 8 KB en PostgreSQL. El motor siempre lee y escribe páginas completas.
- **Sequential Scan**: Lectura exhaustiva de todas las páginas de la tabla de punta a punta.
- **Index Scan**: Búsqueda en el árbol del índice seguida de saltos aleatorios al Heap por cada puntero encontrado.
- **Bitmap Index Scan**: Búsqueda en el índice que construye un mapa de bits en RAM para ordenar las páginas antes de tocarlas, eliminando saltos aleatorios.
- **Index Only Scan**: Consulta que obtiene el 100% de los datos solicitados desde el índice, con 0 accesos al montículo (`Heap Fetches: 0`).
- **Tipping Point**: Umbral de selectividad (5% al 15%) donde el motor prefiere un Seq Scan antes que miles de saltos aleatorios por índice.
- **Shared Buffers**: Memoria RAM asignada para alojar en caché las páginas de tablas e índices (estándar: 25% de la RAM del host).
- **Shared Hit**: Lectura de página resuelta directamente en memoria RAM.
- **Shared Read**: Lectura de página que obligó al motor a ir a buscarla físicamente al disco.
- **Work Mem**: Memoria asignada a cada nodo de ordenamiento o tabla hash en una consulta.
- **MVCC**: Control de Concurrencia Multiversión que evita que las lecturas bloqueen a las escrituras y viceversa.
- **Dead Tuple**: Versión vieja de una fila que quedó obsoleta por un UPDATE o DELETE y ocupa espacio físico en el bloque.

- **Bloat**: Acumulación de tuplas muertas y espacio desperdiciado dentro de los archivos de tablas e índices.
- **VACUUM**: Proceso de limpieza que marca el espacio de tuplas muertas como disponible para reutilización sin bloquear la tabla.
- **VACUUM FULL**: Reescritura física completa de la tabla que devuelve espacio al SO pero bloquea la tabla en modo exclusivo.
- **ANALYZE**: Recolección estadística de la distribución de datos en las columnas para que el optimizador elija el plan correcto.
- **Deadlock**: Bloqueo mutuo circular entre dos o más transacciones que compiten por los mismos recursos. Se aborta automáticamente tras `deadlock_timeout` (1 segundo).
- **Partition Pruning**: Capacidad del optimizador de ignorar particiones irrelevantes basándose en el filtro de la consulta.
- **Vista Materializada**: Tabla en disco que persiste el resultado de una consulta pesada precalculada y admite índices propios para responder en fracciones de milisegundo.